

Video Event System — Server Installation Guide

This guide installs the central server on a clean Raspberry Pi system.

The server provides:

- Video upload API
 - Video storage
 - Browser UI
 - Thumbnail generation
 - Trigger-all support
 - PostgreSQL metadata database
 - Automatic cleanup
 - Node heartbeat monitoring
-

Recommended Hardware

Raspberry Pi

Recommended:

- Raspberry Pi 5 (8 GB)
 - Official Raspberry Pi PSU
 - Active cooling
-

Storage

Recommended:

- USB3 SSD
- 256–512 GB

Filesystem:

- ext4
-

Operating System

Recommended:

- Raspberry Pi OS Lite (64-bit)

Download:

- <https://www.raspberrypi.com/software/>
-

Initial System Setup

Update system:

```
sudo apt update
sudo apt upgrade -y
```

Install useful tools:

```
sudo apt install -y
git
curl
nano
htop
jq
```

Configure Static IP (Recommended)

Recommended:

- Configure static DHCP lease in router

Example:

```
192.168.1.10
```

Verify:

```
hostname -I
```

Install Docker

Install Docker:

```
curl -fsSL https://get.docker.com | sh
```

Add user to Docker group:

```
sudo usermod -aG docker $USER
```

Reboot:

```
sudo reboot
```

Verify Docker

```
docker version
```

```
docker compose version
```

Test:

```
docker run hello-world
```

Enable Docker At Boot

```
sudo systemctl enable docker  
sudo systemctl start docker
```

Prepare SSD Storage

List drives:

```
lsblk
```

Example SSD:

```
/dev/sda
```

Create filesystem if needed:

WARNING: destroys all data on SSD.

```
sudo mkfs.ext4 /dev/sda1
```

Create mount point:

```
sudo mkdir -p /srv/video-storage
```

Get UUID:

```
sudo blkid
```

Example:

```
UUID="1234-5678"
```

Edit fstab:

```
sudo nano /etc/fstab
```

Add:

```
UUID=1234-5678 /srv/video-storage ext4 defaults,nofail 0 2
```

Mount:

```
sudo mount -a
```

Verify:

```
df -h
```

Set permissions:

```
sudo chmod 777 /srv/video-storage
```

Create Project Directory

```
mkdir -p ~/video-server/app  
cd ~/video-server
```

Create docker-compose.yml

File:

```
docker-compose.yml
```

Contents:

```
services:  
  postgres:  
    image: postgres:16  
    environment:  
      POSTGRES_DB: videos  
      POSTGRES_USER: video  
      POSTGRES_PASSWORD: video-password  
    volumes:  
      - postgres-data:/var/lib/postgresql/data  
    restart: unless-stopped  
  
  app:  
    build: ./app  
    ports:  
      - "80:8000"  
    environment:  
      DATABASE_URL: postgresql://video:video-password@postgres:5432/videos  
      VIDEO_STORAGE: /video-storage  
      NODE_TOKEN: your-secret-token  
    volumes:  
      - /srv/video-storage:/video-storage  
    depends_on:  
      - postgres  
    restart: unless-stopped
```

```
volumes:
  postgres-data:
```

Create Application Files

Go to app directory:

```
cd ~/video-server/app
```

requirements.txt

Create:

```
requirements.txt
```

Contents:

```
fastapi
uvicorn[standard]
python-multipart
sqlalchemy
psycopg2-binary
httpx
```

Dockerfile

Create:

```
Dockerfile
```

Contents:

```
FROM python:3.12-slim

RUN apt-get update && apt-get install -y ffmpeg

WORKDIR /app
```

```
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
COPY main.py .  
  
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

main.py

Copy the server application into:

```
~/video-server/app/main.py
```

Build And Start Server

Go to project directory:

```
cd ~/video-server
```

Build and start:

```
docker compose up -d --build
```

Verify Server

Health endpoint:

```
curl http://localhost/api/health
```

Expected:

```
{"status": "ok"}
```

Browser Access

Open browser:

```
http://SERVER_IP/
```

Example:

```
http://192.168.1.10/
```

Status page:

```
http://192.168.1.10/status
```

Configure Node Upload

In node config.yaml:

```
server:
  upload_url: http://192.168.1.10/api/upload
  token: your-secret-token
```

The token must match docker-compose.yml.

Verify Uploads

Trigger node:

```
curl -X POST http://NODE_IP:8080/trigger
```

Verify:

- Clip appears in browser
 - Thumbnail appears
 - Video plays
-

Automatic Cleanup

Create cron job:

```
crontab -e
```

Add:

```
0 3 * * * curl -X POST -H "Authorization: Bearer your-secret-token" http://localhost/api/cleanup
```

Deletes videos older than 30 days.

Useful Commands

Check containers

```
docker compose ps
```

View logs

```
docker compose logs -f app
```

```
docker compose logs -f postgres
```

Restart application

```
docker compose restart app
```

Restart all services

```
docker compose down
```

```
docker compose up -d
```

Verify Storage Usage

```
df -h
```

```
du -sh /srv/video-storage
```

Verify Videos On Disk

```
find /srv/video-storage -name "*.mp4" | head
```

Troubleshooting

Uploads fail

Check:

```
docker compose logs -f app
```

Verify:

- NODE_TOKEN matches node config
 - Server reachable
 - SSD mounted
-

Browser page empty

Check:

```
curl http://localhost/api/videos
```

Node offline

Check:

```
curl http://localhost/api/nodes
```

Verify:

- heartbeat active
 - node reachable
 - firewall disabled
-

No thumbnails

Check FFmpeg inside container:

```
docker compose exec app ffmpeg -version
```

Recommended Backup Strategy

Current design:

- No backups
- Automatic 30-day cleanup

Recommended minimum:

- Backup only configuration files
-

Recommended Maintenance

Monthly:

```
sudo apt update  
sudo apt upgrade -y
```

Update containers:

```
docker compose pull
```

```
docker compose up -d
```

Final Notes

The system is designed to be:

- lightweight
- reliable
- easy to maintain
- LAN-focused
- low complexity
- scalable to multiple nodes

The current architecture supports:

- multiple camera nodes
- grouped trigger events
- browser playback
- thumbnails
- trigger-all
- automatic cleanup
- node heartbeat monitoring
- simultaneous viewing and uploads